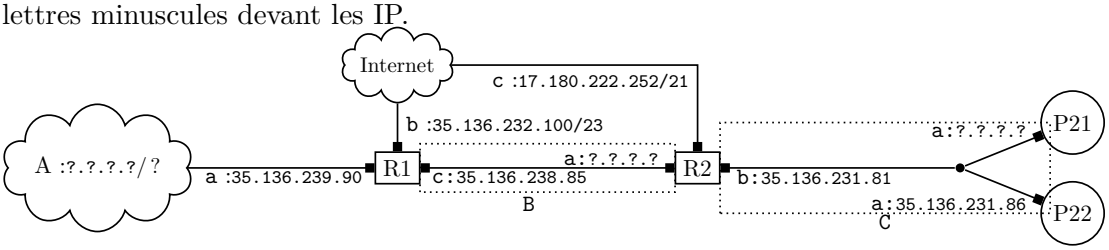
Aucun document ni appareil électronique autorisé. Vous ne devez avoir qu'un stylo et un brouillon. Vous avez 1 heure 30 minutes.

1 Topologie du réseau

On considère le réseau IPv4 constitué de deux routeurs R1 et R2 et 2 machines P21, P22. et un sous réseaux A. Le routeur R1 est connecté à internet avec l'adresse IP `35.136.232.100` avec un routeur à `35.136.232.206` et le masque de sous réseau `255.255.255.0` (23).Le routeur R2 est connecté à internet avec l'adresse IP `17.180.222.252` avec un routeur à `17.180.221.33` et le masque de sous réseau `255.255.248.0` (21). La topologie du réseau est décrite ci- dessous : pour chaque interface l'adresse IPv4 est soit donné soit manquante (`?.?.?.?`). Les nom d'interface sont des lettres minuscules devant les IP.



Les addresses MAC de toutes les interfaces sont les suivantes :
pour R1(a → `36:ae:8b:6f:51:91`, b→`85:3b:2e:a0:51:2e`, c→`76:9b:24:3a:ca:ed`)
pour R2(a→`15:c1:71:04:e5:82`, b→`99:b2:1f:c7:39:0f`), c→`f2:ad:53:7a:6b:3b`)
pour P21(a→`8e:87:4e:5f:d9:3b`) pour P22(a→`68:25:b4:de:4b:f1`)
Les distances pour chacun des liens sont les suivantes :
R1↔A : 9,R1↔Internet :8, R1↔R2 :6, R2↔Internet :15, R2↔P21 :9, R2↔P22 :9, p21↔P22 :9.

2 Réseau et Sous-Réseau

Question 1 Attribué une addresse de réseaux à A, telle qu'il contienne le plus d'addresse possible ? Combien compte t'il d'addresse ?

Question 2 Donnez le plus petit (celui qui contient le moins d'adresse) masque de sous réseaux (sous la forme (X.X.X.X)) possible pour B et C

Question 3 Donnez une adresse IP disponible aux interfaces de R2 dans le sous réseaux B et P21 dans le sous réseaux B

Question 4 Donnez la table de routage de R2 ? On notera les sous réseaux en notation CIDR (`x.x.x.x`/X) ou "default" pour la route par default. On notera chaque routeur par son IP ou "local" si le sous-reseau est localement accessible.

3 Trames

Question 5 L'ordinateur P22 veut charger la page web *http ://www.u-pec.fr/index.html* d'adresse IP `194.214.24.150` en HTTP. Dessinez la quatrième trame échangée entre les deux ordinateurs, en supposant que le client HTTP envoie la requête la plus simple possible. On suppose que le client choisit comme premier numéro de séquence (4109650) et le serveur choisit (2334876). Vous pouvez ignorer les sommes de contrôle, les tailles de fenêtre et la durée de vie.

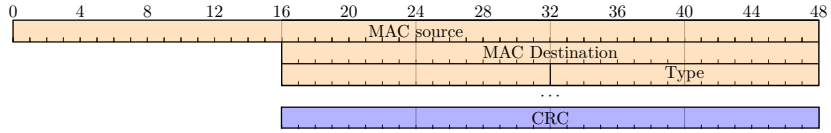
4 Programmation Réseau

On souhaite implémenter un serveur de temps (Network Time Protocol NTP). Ce protocole est basé sur UDP et permet de synchroniser les horloges des ordinateurs sur un réseau. Dans cet exercice, tous les temps sont stockés sous forme de nombres à virgule flottante de 64 bits (double). Le protocole utilisé reçoit un message contenant l'heure d'envoi par le client T_{ec}. Le serveur retient l'heure de réception T_{rs} du message et répond par un message contenant les heures T_{ec}, T_{rs} et T_{ss}, l'heure d'envoi du message par le serveur.

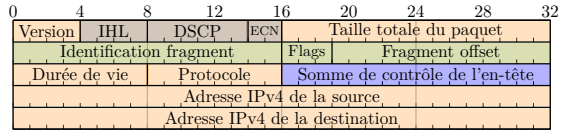
Question 6 Implémentez un tel serveur en Python sur le port 123 sur la machine P22. Vous pourrez utiliser la liste de fonctions à la fin de ce sujet. Notamment, pour écrire un nombre en virgule flottante dans un **bytearray**, vous pouvez utiliser la fonction struct.pack(**format**, v1, v2, ...) qui a la même syntaxe que printf en utilisant 'd' pour double. La fonction struct.unpack(**format**, **bytearray**) fait l'opération inverse.

5 Référence

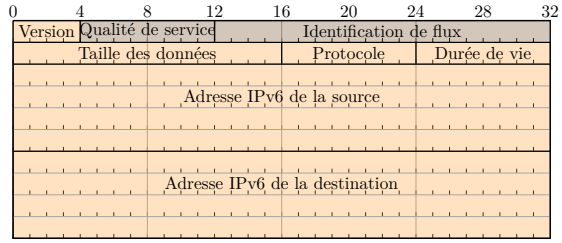
En-tête et fin Ethernet :



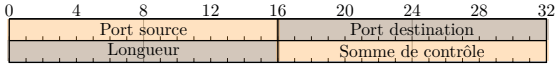
En-tête IPv4 :



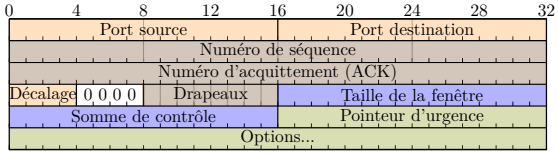
En-tête IPv6 :



En-tête UDP :



En-tête TCP :



Type Ethernet : IPv4 :0x0800, ARP : 0x0806, IPv6 : 0x86DD.
Numéro de protocole : ICMP :1, TCP :6, UDP :17.

Ci-dessous les fonctions Python dont vous pourriez avoir besoin :

- time.time() renvoie l'heure courante comme un nombre flottant.
- socket.socket(domaine, **type**) Construit un socket.

- socket.bind(adresse) Lie un socket à une adresse.
- socket.listen() Le socket se met à attendre des connexions.
- socket.accept() Accepte une connexion sur un socket.
- socket.close() Ferme la connexion d'un socket.
- socket.send(msg) Envoie le **bytearray** msg sur le socket et renvoie le nombre d'octets effectivement envoyés.
- socket.sendto(msg, adresse) Envoie le **bytearray** msg sur le socket à l'adresse spécifiée et renvoie le nombre d'octets effectivement envoyés.
- socket.sendall(msg) Envoie tout le **bytearray** msg sur le socket potentiellement en plusieurs paquets.
- socket.recv(taille) Lit au plus taille octets sur le socket et les renvoie comme un bytearray.
- socket.recvfrom(taille) Lit au plus taille octets sur le socket et renvoie un couple avec les données et l'adresse de l'expéditeur du message.
- **len**(ba) Renvoie le nombre d'octets dans le **bytearray** ba.
- **int**(str) Convertit une chaîne de caractères en entier.